



Excelencia que trasciende

DELVALLE
GRUPO EDUCATIVO

Laboratorio 2 Redes:

Esquemas de detección y corrección de errores

Nicolle Gordillo - 22246
Giovanni Santos - 22523

1. Descripción de la práctica

El objetivo de esta práctica fue construir una aplicación de intercambio de mensajes entre un cajero automático y el servidor de una entidad financiera, sobre un canal de transmisión no confiable, y comparar experimentalmente dos familias de esquemas de integridad: los de corrección de errores y los de detección de errores.

La aplicación se organizó en la arquitectura de capas solicitada. El emisor (cajero automático) se implementó en C++ y el receptor (servidor bancario) en Python. La comunicación se realiza mediante sockets TCP sobre el puerto 5050.

1.1 Capas y servicios implementados

Capa	Servicios	Emisor (C++)	Receptor (Python)
Aplicación	solicitar_mensaje, mostrar_mensaje	client.cpp	server.py
Presentación	codificar_mensaje, decodificar_mensaje	SenderCapaPresentacion	ReceptorCapaPresentacion
Enlace	calcular_integridad, verificar_integridad, corregir_mensaje	SenderHamming, SenderFletcher	ReceptorHamming, ReceptorFletcher
Ruido	aplicar_ruido	CapaRuido	— (solo emisor)
Transmisión	enviar_informacion, recibir_informacion	enviarInformacion()	recibir_informacion()

La capa de aplicación solicita, antes de la primera transacción, el algoritmo de integridad que se utilizará y la tasa de error del canal. La capa de presentación codifica cada carácter del mensaje JSON en ASCII binario de 8 bits. La capa de enlace calcula y concatena la información de redundancia. La capa de ruido recorre la trama completa (incluidos los bits de redundancia) y voltea cada bit de forma independiente con la probabilidad indicada. Finalmente, la capa de transmisión envía la trama por el socket.

Para que el receptor sepa qué algoritmo debe aplicar, cada paquete se antepone con una cabecera corta (HAM, FLE8, FLE16 o FLE32) separada por el carácter |, y se delimita con un salto de línea. Esta cabecera es metadato de protocolo y no se somete al ruido: representa la información de control que en una implementación real viajaría en un campo de encabezado protegido de forma independiente. El delimitador de salto de línea resuelve además el problema de segmentación de TCP, que no preserva los límites de mensaje.

1.2 Algoritmos implementados

Se implementó un algoritmo de cada tipo, uno por integrante:

- Corrección — Código de Hamming, para cualquier código (n, m) que cumpla $m + r + 1 \leq 2^n$. Emisor en C++ (algHammingSnd.hpp) y receptor en Python (alghammingRe.py).
- Detección — Fletcher checksum, con tamaño de bloque configurable de 8, 16 o 32 bits, lo que produce checksums de 16, 32 y 64 bits respectivamente. Emisor en C++

(algFletcherSnd.hpp) y receptor en Python (algFletcherRe.py). Cuando la longitud del mensaje no es múltiplo del tamaño de bloque se agregan ceros de relleno al final.

El cálculo de Fletcher acumula dos sumas sobre los bloques de la trama, ambas en aritmética módulo $2^b - 1$, donde b es el tamaño de bloque. La primera suma acumula el valor de cada bloque; la segunda acumula el valor corriente de la primera, lo que hace que el resultado dependa del orden de los bloques y no solo de su contenido. El checksum final es la concatenación de ambas sumas.

1.3 Validación cruzada entre lenguajes

Dado que el emisor y el receptor están escritos en lenguajes distintos, se verificó que ambas implementaciones fueran equivalentes bit a bit. Se generaron 200 tramas aleatorias de longitud variable (1 a 300 bits) con los tres tamaños de bloque y se comparó la salida del emisor en C++ contra una implementación espejo en Python. Las 200 comparaciones coincidieron exactamente, y en todos los casos el receptor recuperó los datos originales sobre un canal sin ruido.

2. Resultados

Las pruebas se realizaron simulando la cadena completa emisor $\rightarrow$ ruido $\rightarrow$ receptor, lo que permitió ejecutar miles de repeticiones por configuración. Se variaron cuatro factores: el tamaño del mensaje (5, 20, 50 y 100 caracteres), la probabilidad de error por bit (de 0.001 a 0.3), el algoritmo utilizado y el overhead. Cada punto de la malla principal corresponde a 300 repeticiones, con semilla fija para garantizar reproducibilidad.

2.1 Métricas utilizadas

Comparar un algoritmo de corrección con uno de detección exige distinguir dos cosas que suelen confundirse:

- Tasa de entrega: porcentaje de tramas cuyo contenido llegó íntegro a la capa de aplicación. Para Hamming incluye las tramas corregidas con éxito; para Fletcher solo las que no sufrieron ningún cambio, ya que el algoritmo no corrige.
- Tasa de integridad: porcentaje de tramas que no terminaron en una corrupción silenciosa, es decir, que llegaron correctas o bien fueron rechazadas explícitamente. Su complemento es el fallo silencioso: datos alterados que el receptor aceptó como válidos.

Esta segunda métrica es la relevante en el contexto del laboratorio. Un cajero automático que rechaza una transacción provoca una molestia, pero uno que procesa un retiro por un monto distinto al solicitado provoca un problema mucho mayor.

2.2 Resultados del código de Hamming

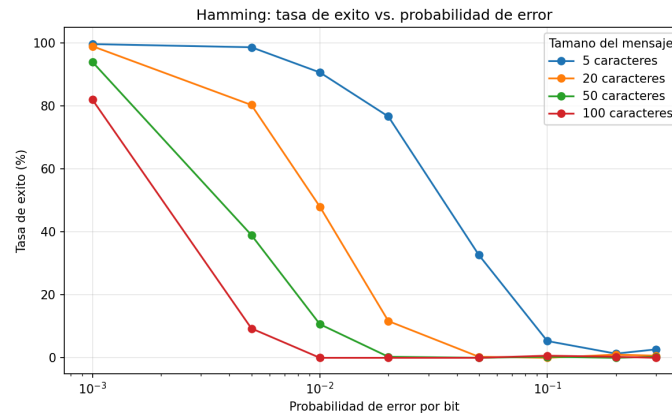


Figura 1: Tasa de éxito vs probabilidad de error con Hamming

Según la figura 1, se observa que la tasa de éxito cae de forma abrupta cuando se tiene un salto de $p=0.1$ indicando que hamming corrige de forma confiable cuando se tiene como máximo 1 bit erróneo por bloque codificado. Adicionalmente, se puede observar que el tamaño del mensaje hace que la caída sea más abrupta. Específicamente, cuando $p=0.01$, el mensaje de 5 caracteres da 90.67% de éxito, pero el de 100 caracteres muestra 0.0%. Como resultado, se observa el comportamiento que mediante mensajes más largos, se tienen bits más expuestos al ruido y por lo tanto, mayor probabilidad de errores caigan en el mismo bloque de paridad.

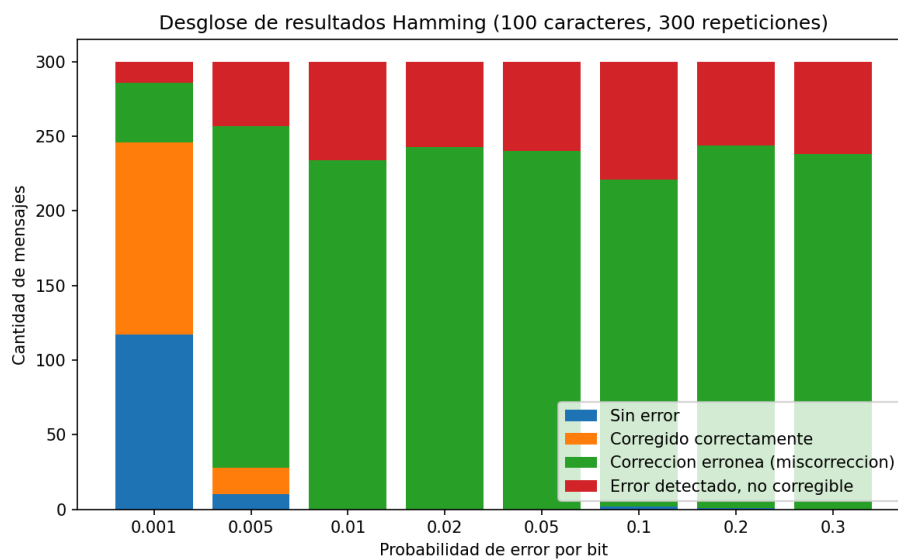


Figura 2: Desglose de resultados Hamming

Se puede observar con la figura 2, que a partir de $p=0.02$ para 100 caracteres se tiene el resultado predominante de de miscorrección. 243 de 300 casos en $p=0.02$ muestran esta tendencia que muestra un error dominante en el algoritmo de hamming. De esta manera, se tiene la necesidad de implementar protocolos reales de seguridad además de la corrección básica.

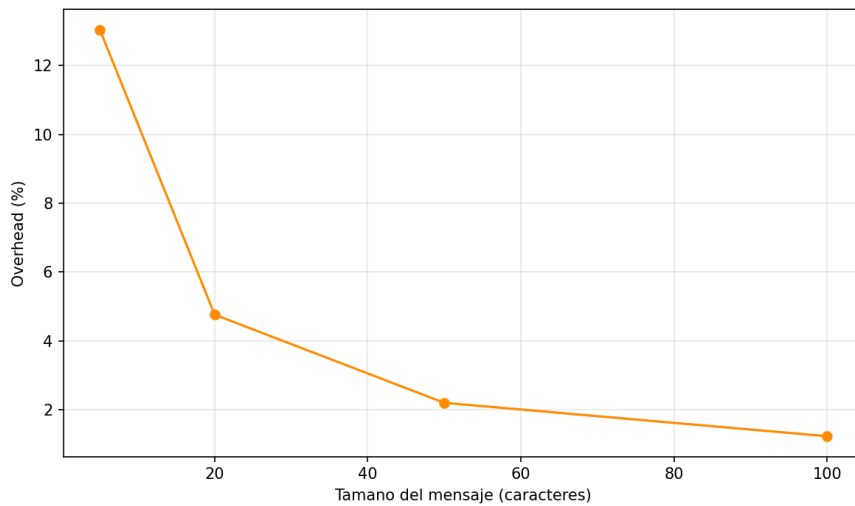


Figura 3: Overhead de Hamming vs tamaño del mensaje

En la figura 3, se puede observar que el overhead baja de 13.04% en 5 caracteres a 1.23% en 100 caracteres. Se puede establecer que Hamming es más eficiente en mensajes largos en términos de bits de redundancia, sin embargo, se tiene un trade-off con la confiabilidad.

2.3 Capacidad de entrega: comparación entre ambos esquemas

Figura 1. Capacidad de entrega: correccion vs. deteccion

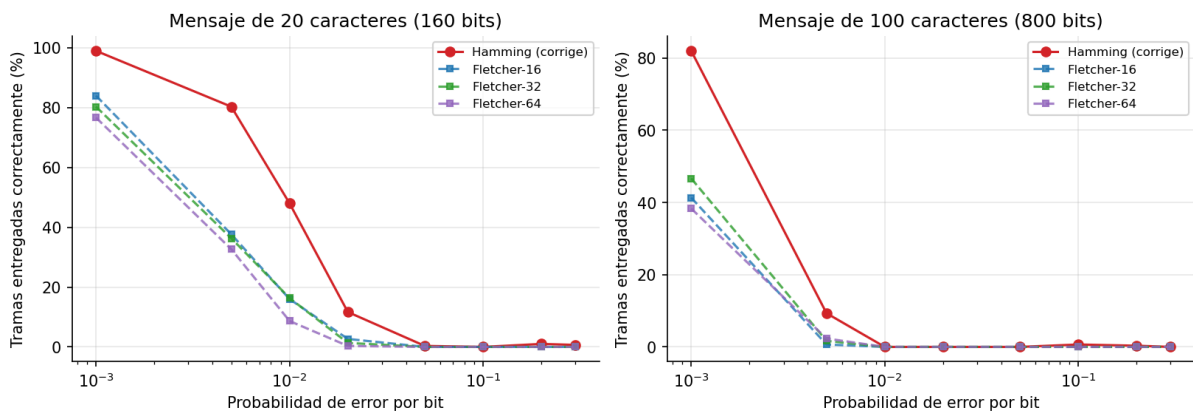


Figura 4. Porcentaje de tramas entregadas correctamente a la capa de aplicación, en función de la probabilidad de error por bit (escala logarítmica).

Hamming entrega consistentemente más tramas que Fletcher en el régimen de errores bajos, lo cual es esperable: sobre un mensaje de 20 caracteres con $p = 0.005$, Hamming entrega el 80.3 % de las tramas frente al 37.7 % de Fletcher-16. La diferencia se explica enteramente por la capacidad de corrección, ya que en ese régimen la mayoría de las tramas afectadas contienen un solo bit erróneo.

A partir de $p \approx 0.02$ ambas curvas colapsan hacia cero. Con mensajes de 100 caracteres la trama supera los 800 bits, de modo que una probabilidad de 0.02 produce en promedio más de 16 bits alterados y ninguno de los dos esquemas puede entregar la trama intacta.

2.4 Fallos silenciosos

Figura 2. Fallos silenciosos: el riesgo de corregir a ciegas

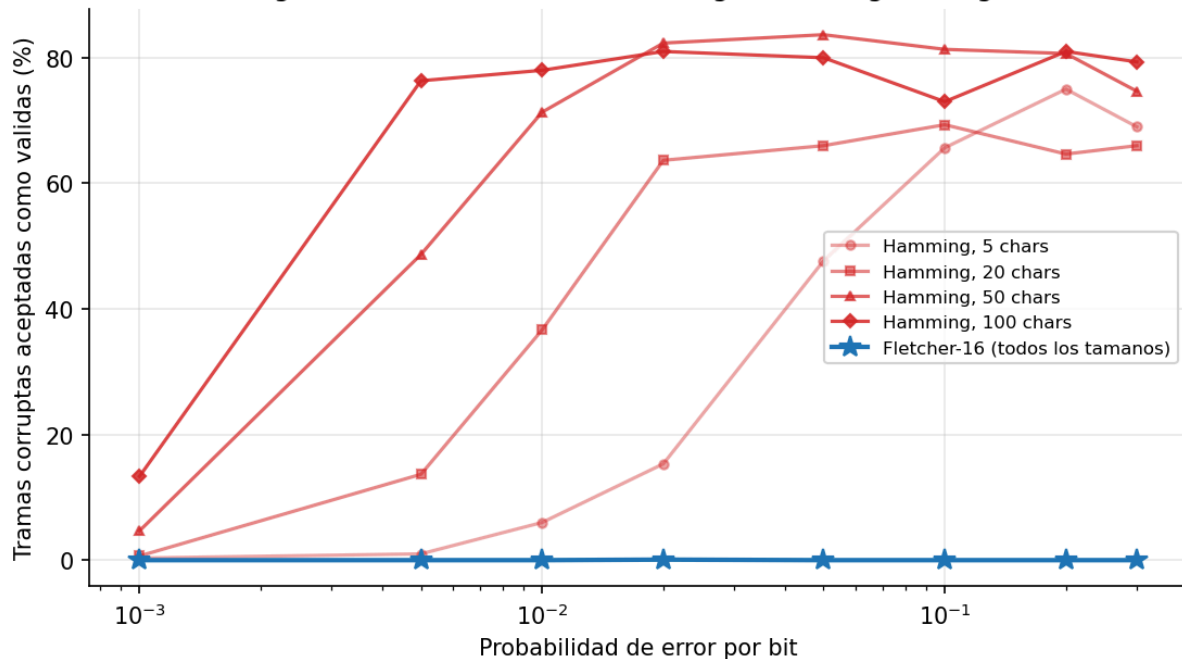


Figura 5. Porcentaje de tramas corruptas que el receptor aceptó como válidas. Fletcher permanece indistinguible de cero en toda la malla.

Este es el resultado más marcado del experimento. Sobre el total de la malla de pruebas de Fletcher se registraron 24 589 tramas con error, de las cuales una sola no fue detectada, para una tasa de detección global del 99.996 %. En el caso de Hamming, 5 250 de 9 600 tramas, o sea el 54.7 %, fueron entregadas a la capa de aplicación con datos incorrectos sin que el receptor lo advirtiera.

La causa es estructural. Un código de Hamming sin bit de paridad global tiene distancia mínima 3, lo que le permite corregir un error pero no distinguir entre uno y dos. Cuando ocurren dos errores el síndrome apunta a una tercera posición inocente, el receptor la corrige y entrega una trama con tres bits equivocados creyendo haberla reparado. El algoritmo no falla por no detectar el error: falla por estar seguro de haberlo arreglado.

2.5 Robustez según la cantidad de bits alterados

Figura 4. Robustez segun la cantidad de bits alterados (50 caracteres, 5000 repeticiones por punto)

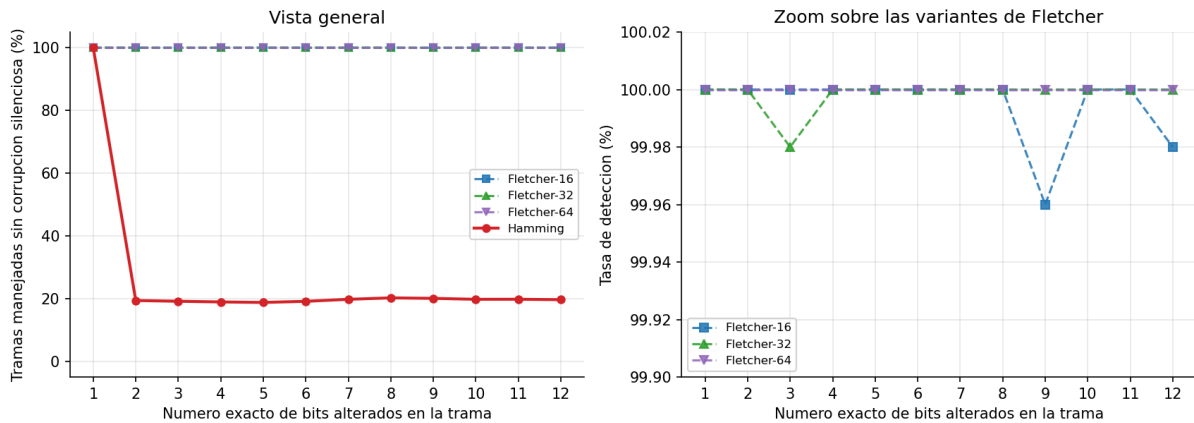


Figura 6. Comportamiento cuando se altera un número exacto y controlado de bits. 5 000 repeticiones por punto sobre mensajes de 50 caracteres.

Al controlar el número exacto de bits alterados el contraste se vuelve nítido. Con un solo bit erróneo ambos algoritmos alcanzan el 100 %: Hamming lo corrige y Fletcher lo detecta. A partir de dos bits el comportamiento de Hamming cae al 19.4 % y se mantiene en torno al 19–20 % sin importar cuántos bits se alteren. Ese 19 % residual corresponde a los casos en que el síndrome calculado excede la longitud de la trama, única situación en la que el receptor reconoce que no puede corregir.

Fletcher, en cambio, se mantiene sobre el 99.96 % en todo el rango probado. El panel derecho muestra que las diferencias entre los tres tamaños de bloque son del orden de dos o tres tramas en cinco mil, consistentes con la probabilidad teórica de colisión de un checksum de 16 bits (aproximadamente 1 en 65 536).

2.6 Overhead

Figura 5. Costo en redundancia de cada algoritmo

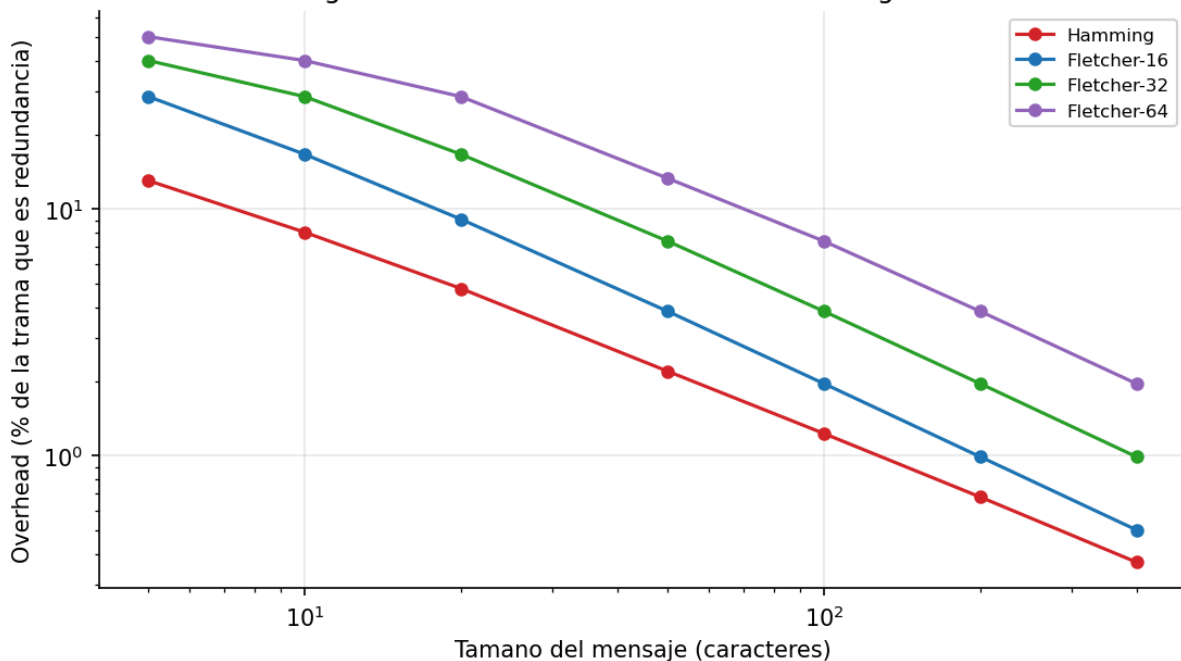


Figura 7. Costo en redundancia de cada algoritmo

El overhead de los dos algoritmos escala de forma cualitativamente distinta. Hamming requiere r bits de paridad para m bits de datos según $m + r + 1 \leq 2^r$, es decir un crecimiento logarítmico: cada duplicación del mensaje agrega aproximadamente un bit de paridad. Fletcher tiene un costo fijo de 2b bits independientemente del tamaño del mensaje.

Tamaño	Hamming	Fletcher-16	Fletcher-32	Fletcher-64
5 caracteres	13.04 %	28.57 %	40.00 %	50.00 %
20 caracteres	4.76 %	9.09 %	16.67 %	28.57 %
50 caracteres	2.20 %	3.85 %	7.41 %	13.33 %
100 caracteres	1.23 %	1.96 %	3.85 %	7.41 %
400 caracteres	0.37 %	0.50 %	0.99 %	1.96 %

En mensajes cortos Hamming resulta claramente más económico, mientras que Fletcher-64 llega a consumir la mitad de la trama. La ventaja de Hamming se diluye conforme el mensaje crece: con 400 caracteres la diferencia frente a Fletcher-16 es de apenas 0.13 puntos porcentuales, un costo despreciable frente a la diferencia en capacidad de detección.

2.7 Integridad sobre la malla completa

Figura 6. Integridad: tramas que no terminaron en corrupción silenciosa

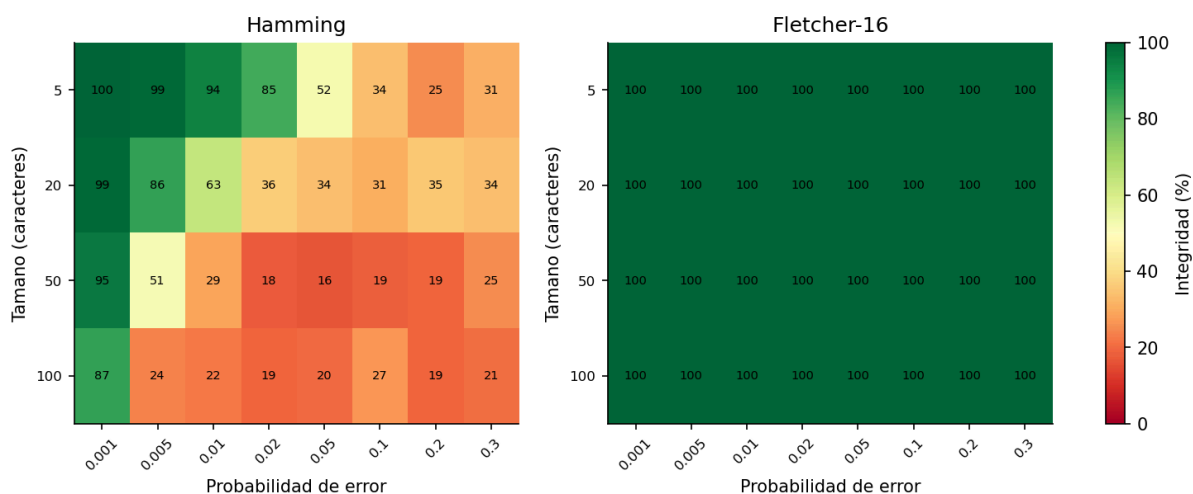


Figura 8. Porcentaje de tramas que no terminaron en corrupción silenciosa, para cada combinación de tamaño y probabilidad de error.

El mapa de calor resume el comportamiento sobre toda la malla. Fletcher se mantiene en verde en las 32 celdas; el único valor por debajo de 100 corresponde a una trama entre trescientas. Hamming se degrada rápidamente conforme aumentan tanto el tamaño del mensaje como la tasa de error, porque ambos factores incrementan la probabilidad de que una trama contenga dos o más bits alterados.

3. Discusión

Cómo se observa en la figura 1, hamming corrige de forma confiable cuando se tiene un bit erróneo. Específicamente, disminuye la tasa de éxito de 99% con $p=0.001$ a 5% con $p=0.1$, indicando que la confiabilidad disminuye significativamente. Por otro lado, en la figura 2, se puede notar que a partir de $p=0.02$ para 100 caracteres, domina la corrección errónea en donde el algoritmo indica falsamente una corrección, indicando que Hamming por sí solo no es suficiente para detectar todos los errores en un sistema real. Adicionalmente, según la figura 3, se observa que Hamming es eficiente en disminuir el overhead con mensajes largos, detectando la redundancia. Sin embargo, vimos que mensajes largos disminuyen la confiabilidad, por lo que se tiene que reconsiderar el tamaño de mensajes eficientes para el algoritmo.

Ninguno de los algoritmos es explícitamente mejor que el otro. Bajo el criterio de cuántos mensajes llegan sin retransmisión, Hamming es superior con errores bajos: con 20 caracteres y $p = 0.005$ entrega el 80.3 % de las tramas frente al 37.7 % de Fletcher-16. Bajo el criterio de qué tan confiable es lo que se entrega, la relación se invierte: Fletcher detectó 24 588 de 24 589 tramas alteradas, mientras que Hamming entregó datos incorrectos sin advertirlo en el 54.7 % de las tramas afectadas. Para una aplicación bancaria el criterio más importante es el segundo, porque una trama corrupta aceptada significa autorizar un retiro por un monto o contra una tarjeta equivocados.

También, ambos responden diferente cuando el canal se degrada. La detección de Fletcher se mantiene sobre el 99.96 % entre uno y doce bits alterados (Figura 6), porque no depende de cuántos bits cambien sino de la probabilidad de que la alteración produzca el mismo checksum. En cambio, para Hamming, la capacidad está fijada en un error por trama y, superado ese umbral, agrega un error adicional al corregir una posición inocente, de ahí el escalón del 100 % al 19.4 % entre uno y dos bits. Conviene matizar que el modelo de ruido voltea cada bit de forma independiente, lo cual favorece a Fletcher, ya que los canales reales producen errores en ráfagas.

- ¿Cuándo es mejor utilizar un algoritmo de detección de errores en lugar de uno de corrección de errores?

Comentario grupal sobre el tema (opcional)

Conclusiones

- Hamming es más eficiente con $p \leq 0.001$, siempre que el mensaje sea corto: el umbral depende del producto de la probabilidad por la longitud de la trama, no de la probabilidad sola. Con 100 caracteres a esa misma probabilidad ya falla silenciosamente en el 13.3 % de los casos.
- Fletcher detectó 24 588 de 24 589 tramas alteradas (99.996 %), con un comportamiento estable frente a la probabilidad de error y al número de bits alterados. Su punto débil no es la detección sino la ausencia de recuperación: toda trama afectada debe retransmitirse.
- Hamming corrigió el 100 % de los errores de un solo bit, pero entregó datos incorrectos sin advertirlo en el 54.7 % de las tramas afectadas. Su distancia mínima de 3 le impide distinguir un error de dos, y ante dos errores no falla de forma segura sino que agrega un tercero.
- Para una aplicación financiera la métrica decisiva no es cuántos mensajes llegan sino cuántos llegan mal sin ser advertidos, por lo que el esquema de detección es la elección adecuada.

Esto coincide con el diseño de los protocolos de red reales, que detectan y descartan en lugar de corregir.

Citas y Referencias

Fletcher, J. G. (1982). An arithmetic checksum for serial transmissions. *IEEE Transactions on Communications*, 30(1), 247–252.

GeeksforGeeks. (2025, July 12). *Hamming Code implementation in Python*. GeeksforGeeks.

<https://www.geeksforgeeks.org/python/hamming-code-implementation-in-python/>

Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160.

Maxino, T. C., & Koopman, P. J. (2009). The effectiveness of checksums for embedded control networks. *IEEE Transactions on Dependable and Secure Computing*, 6(1), 59–72.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer networks* (5th ed.). Prentice Hall.

Zweig, J., & Partridge, C. (1990). TCP alternate checksum options (RFC 1146). Internet Engineering Task Force.